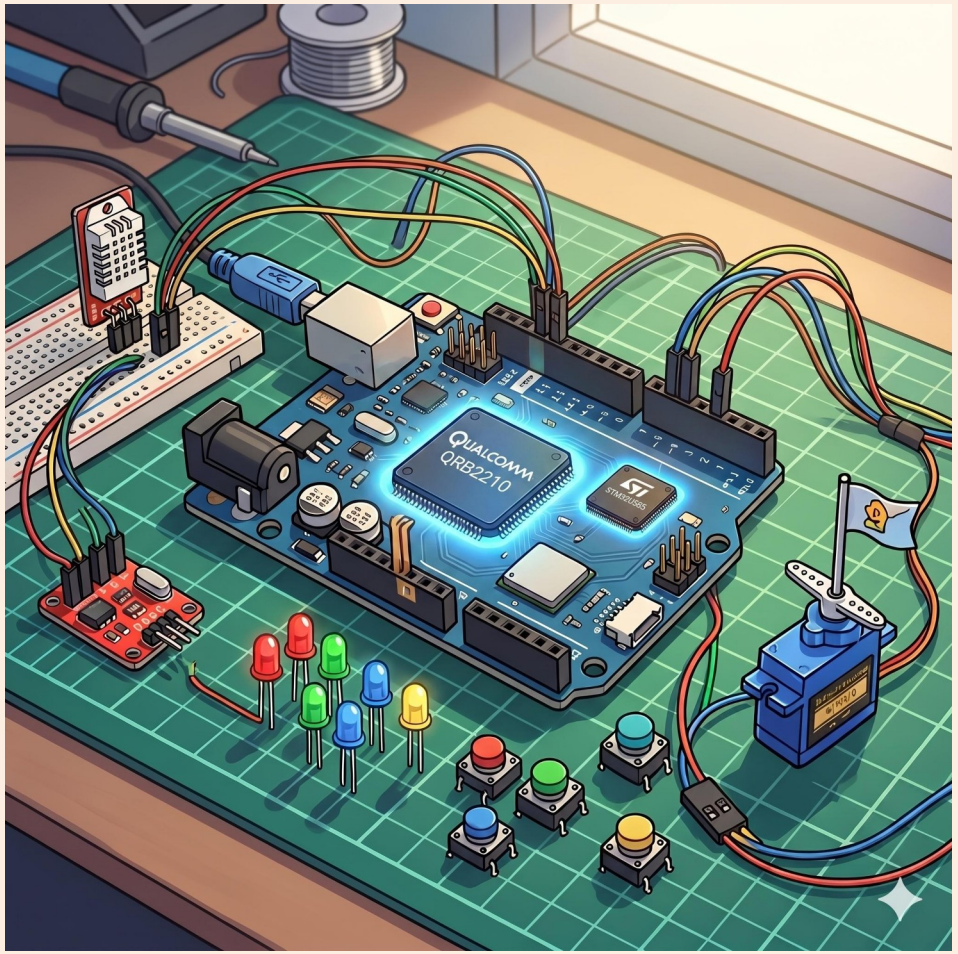
CHAPTER 2 · IESTI05 EDGE AI ENGINEERING

Generative AI at the Edge

Running small language models with llama.cpp

> No internet · no API keys · no cloud · nothing leaves the board

Marcelo Rovai · UNIFEI



AIEng4D

WHERE WE ARE GOING

A language model answering on 3.1 watts, with the network unplugged.

About 6 to 7 tokens per second, ~54 °C, no heatsink and no fan.



THE ROUTE

Six stages, one local model at the end

1. **Constraints** — Memory, storage, model choice and quantization — decided before you download.
2. **Build** — Swap, build tools, a shallow clone, CMake, and 30 minutes of four cores at 100%.
3. **llama-cli** — One prompt, one answer — and the one flag that decides if this board is usable.
4. **Server** — Load the model once, expose an OpenAI-compatible HTTP API, query it with curl.
5. **Python** — The openai client library, then raw HTTP, so you see what the library does.
6. **Limits** — Measured throughput, the four failure modes, and when 0.8B stops being enough.

OBJECTIVES

Three ways to talk to a local model

STAGE 3

llama-cli

One-shot prompts. No server, no extra process.

The fastest way to sanity-check a model before building anything on top of it.

STAGE 4

llama-server

An OpenAI-compatible HTTP API.

Loads the GGUF into RAM once and serves any client that can speak JSON.

STAGE 5

Python

The standard openai client library, then raw HTTP — so you can see what the library is doing underneath.

Prerequisite: Chapter 1. You need SSH access to the board. Nothing else is assumed.

STAGE 1 OF 6

What is realistic on this board

1

Memory, storage, model choice and quantization — decided before you download anything.

FRAMING

Why small language models change the picture

WHAT WE TAUGHT BEFORE

Classic TinyML

One model, one task. A wake-word detector. An image classifier.

Milliseconds of latency, kilobytes of RAM — and a retraining cycle for every new task.

WHAT CHANGES HERE

A small language model

One sub-1B model can classify, summarise, reformat, translate and reason about structured input.

Seconds of latency, and it needs Linux and a couple of GB of RAM.

'Small' means roughly 100 M to 7 B parameters — pruned, distilled and quantized to run CPU-only.

THE TRADE

What you pay, and what you get

seconds

not milliseconds

the latency you are buying with

≥ 2 GB RAM

+ Linux

the platform floor

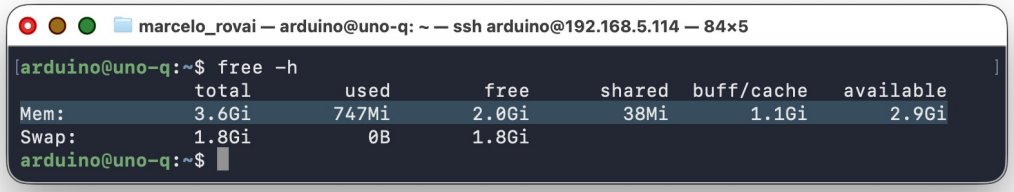
one model

many tasks

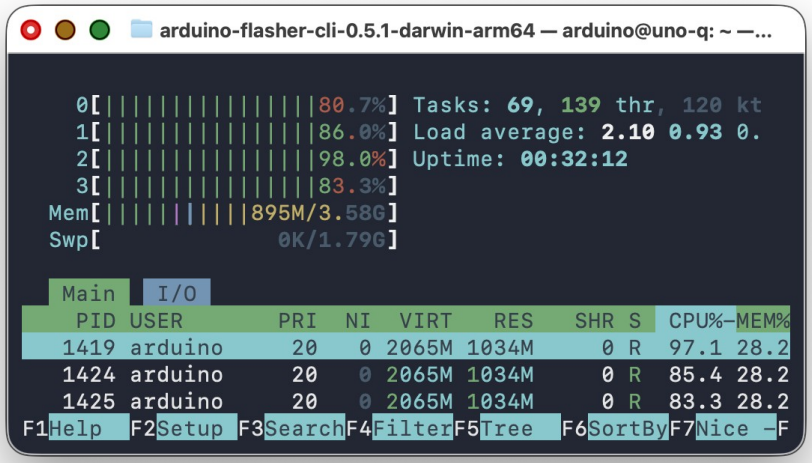
what you get in return

CONSTRAINT 1

The memory reality on a 4 GB board



free -h after boot, nothing running



htop while a 0.8B Q8 model generates

Model + context ≈ 1.1 GB, leaving ~1.7 GB. The 2 GB board needs a smaller model — SmolLM2-135M or 360M.

CONSTRAINT 2

Check your partition layout first

```
$ df -h /home/arduino
# do this BEFORE you download anything
```

> Two layouts are in the wild. Which one you have decides whether you need the shallow clone and the build-tree cleanup at all.

CONSTRAINT 2

The two layouts you might have

OLDER IMAGES

One 9.8 GB partition

/ and /home share it, with about 830 MB free after boot.

Disk management matters: use the shallow clone, and delete the build tree afterwards.

NEWER IMAGES

/home on its own 18 GB

Source, models and projects all land there; the 9.8 GB root stays untouched.

You can skip the cleanup steps entirely.

You need at least 5 GB free to complete the build and download a model.

MODEL CHOICE

Why Qwen3.5-0.8B

Qwen3.5 Family Comparison on Shared Benchmarks

	397B-A17B	122B-A10B	35B-A3B	27B	9B	4B	2B	0.8B
Overall	100%	97%	95%	97%	90%	85%	72%	54%
Text Overall	100%	97%	94%	97%	90%	84%	64%	41%
General Agent	100%	95%	93%	93%	91%	81%	58%	24%
Long Context	100%	96%	89%	96%	90%	81%	49%	24%
Instruction Following	100%	97%	93%	98%	88%	82%	63%	37%
Multilingualism	100%	98%	95%	98%	90%	84%	64%	46%
Knowledge	100%	98%	96%	97%	92%	88%	73%	49%
Visual Overall	100%	97%	95%	97%	90%	86%	76%	62%
STEM and Puzzle	100%	93%	89%	94%	79%	76%	61%	43%
Medical VQA	100%	99%	94%	95%	87%	81%	72%	62%
Video Understanding	100%	99%	97%	98%	94%	90%	78%	62%
Visual Agent	100%	107%	105%	107%	99%	92%	83%	71%
Spatial Intelligence	100%	97%	96%	97%	91%	87%	77%	67%
General VQA	100%	97%	96%	95%	92%	88%	81%	68%
Text Recognition and Document Understanding	100%	98%	98%	98%	95%	92%	85%	71%

≤ 80% ≥ 103%

Color scale: 80%–103%. Values below 80% or above 103% are clipped.

The Qwen3.5 Small family

Released February 2026 · Apache 2.0 · 201 languages · designed for edge inference.

OPTIONS

Candidate models for this board

Model	Params	Size	Notes
`SmolLM2-135M-Instruct Q4_K_M`	135 M	~95 MB	Very fast, limited reasoning. Routing and classification only.
`SmolLM2-360M-Instruct Q4_K_M`	360 M	~230 MB	Balanced. The fallback if 0.8B is too tight.
`Qwen3.5-0.8B Q8_0`	800 M	~800 MB	The primary choice. Best quality-per-parameter.
`Qwen3.5-2B UD-Q4_K_XL`	2 B	~1.34 GB	Excellent, about half the speed of the 0.8B.

Have the fallback ready: a student on a 2 GB board or short of disk stays in the lesson with SmolLM2-360M instead of stuck on a download.

GETTING THE WEIGHTS

Download a GGUF

```
$ mkdir -p ~/models/Qwen3.5-0.8B-GGUF
$ cd ~/models/Qwen3.5-0.8B-GGUF

$ wget -c "huggingface.co/unsloth/Qwen3.5-0.8B-GGUF/\
       resolve/main/Qwen3.5-0.8B-Q8_0.gguf"
```

> Either Q8_0 works: Unsloth's (used here) or Bartowski's Qwen_Qwen3.5-0.8B-Q8_0.gguf (used in the book). Point --model at the file you downloaded.

CONSTRAINT 3

At sub-1B scale, Q4 is aggressive.

The quantization error compounds in a smaller model — there is less redundancy to absorb it. At 7B, Q4 is nearly free. At 0.8B it removes information the model cannot afford to lose.

CONSTRAINT 3

What to do about it

1 Use Q8_0 for the 0.8B

~800 MB. Meaningfully better than Q4 — and what this chapter uses.

2 Use UD-Q4_K_XL for the 2B

Unsloth Dynamic upcasts critical layers to 8 or 16 bits, near Q4 file size (~1.34 GB).

3 Q4_K_M only to test the workflow

~550 MB. Proves the pipeline fits, but the 0.8B output is noticeably weaker.

4 Prefer min_p over top_p

min_p 0.05 with temp 0.7 adapts to the actual distribution; top_p at low temperature makes repetition almost inevitable at this size.

> If you must ship Q4: strong few-shot prompting, response_format json_object, presence_penalty 1.5, low max_tokens.

STAGE 2 OF 6

Building llama.cpp from source

2

Swap, build tools, a shallow clone, CMake — and 30 minutes of four A53 cores at 100%.

PREPARATION

Check the ground first

```
$ uname -m          # aarch64
$ free -h           # ~3.6 Gi total, Swap: ~1.8 Gi
$ df -h /           # root partition space
```

If Swap shows 0 B, add it on the next slide before you build. The OOM killer during model load is what you are protecting against.



free -h — swap is present

PREPARATION

Add 2 GB of swap, if it is missing

```
$ sudo fallocate -l 2G /home/swapfile
$ sudo chmod 600 /home/swapfile
$ sudo mkswap /home/swapfile
$ sudo swapon /home/swapfile

# make it survive a reboot
$ echo '/home/swapfile none swap sw 0 0' | \
      sudo tee -a /etc/fstab
```

> Only needed on older images. Newer ones ship with swap already configured.

PREPARATION

Install the build tools

```
$ sudo apt update
$ sudo apt install -y build-essential cmake git \
      pkg-config libssl-dev
```

> libssl-dev lets llama-cli and llama-server download a GGUF themselves. Older recipes use libcurl, which current builds no longer need.

THE BUILD

Shallow clone, then CMake

```
$ cd /home/arduino
$ git clone --depth 1 https://github.com/ggml-org/llama.cpp
$ cd llama.cpp

$ cmake -B build \
    -DGGML_NATIVE=ON \
    -DCMAKE_BUILD_TYPE=Release
$ cmake --build build --config Release -j4 \
    --target llama-cli llama-server
```

~30 minutes with -j4 (a full build is ~46). If the board freezes or the build dies, drop to -j2 — slower, but it stays under the OOM threshold.

THE BUILD

What those four flags are doing

1

--depth 1

A full clone pulls ~400 MB of git history you will never read. Shallow is ~220 MB.

2

--target llama-cli llama-server

Builds only the two tools we use: ~29 minutes instead of ~46.

3

-DGGML_NATIVE=ON

Detects the A53's NEON SIMD capabilities and emits the right CPU intrinsics.

4

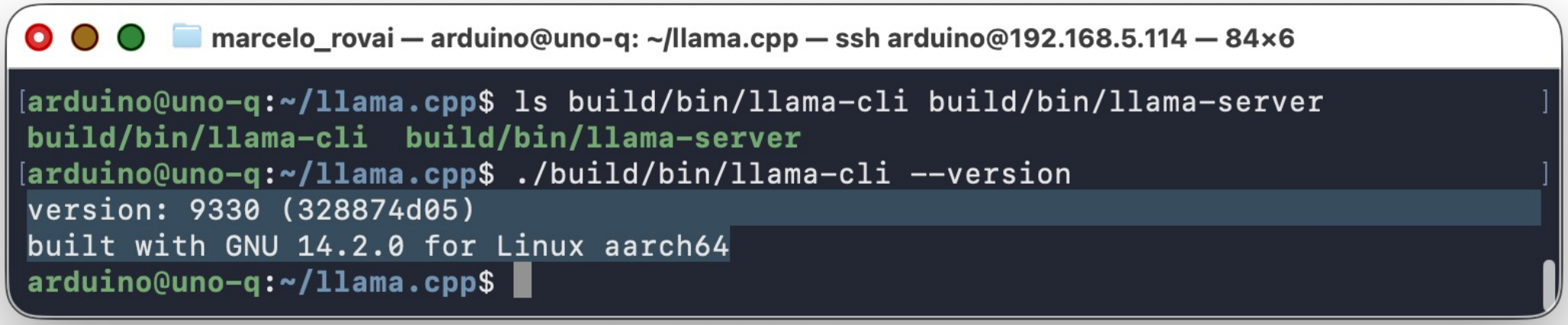
-j4

All four cores. Drop to -j2 if the build runs out of memory.

> Verify when it finishes: ls build/bin/llama-cli build/bin/llama-server

VERIFY

You should have two binaries



llama-cli --version — the build worked

SHORTCUT

Or skip the build entirely.

llama.cpp publishes pre-built aarch64 Linux binaries on its GitHub Releases page. Download the tarball, extract llama-server and llama-cli, and you're done.



HOUSEKEEPING

Reclaim ~340 MB after the build

```
$ cd /home/arduino && mkdir -p llama-runtime
$ cp llama.cpp/build/bin/llama-server llama-runtime/
$ cp llama.cpp/build/bin/llama-cli    llama-runtime/
$ cp llama.cpp/build/bin/*.so*        llama-runtime/

# verify it stands alone BEFORE deleting anything
$ cd llama-runtime && LD_LIBRARY_PATH=. ./llama-cli --version

$ rm -rf /home/arduino/llama.cpp     # saves ~340 MB
```

Only on the older single-partition image. The LD_LIBRARY_PATH test is not optional — find out which shared objects are copied while the source tree is still present.

STAGE 3 OF 6

llama-cli: the sanity check

3

One prompt, one answer, no server — and one flag that decides whether the board is usable.

THE CRITICAL FLAG

Turn thinking mode off. Always.

Qwen3.5 is a hybrid reasoning model and defaults to thinking ON. On this hardware that means 30–60 seconds of internal chain-of-thought before any output — and frequent loops that never reach a conclusion.

THE CRITICAL FLAG

The two flags that turn it off

```
--reasoning off  --reasoning-budget 0
```

> The older --chat-template-kwargs '{"enable_thinking":false}' is deprecated and emits a warning. Replace it wherever you find it.

SAMPLING

Recommended parameters for non-thinking mode

Parameter	Value	Purpose
`temperature`	0.7 – 1.0	Moderate creativity for general tasks
`top_p`	0.8	Nucleus sampling
`top_k`	20	Limit the token pool
`min_p`	0.0	Disabled by default — but see the note on small models
`presence_penalty`	1.5	Prevent repetition loops — critical at this size
`repetition_penalty`	1.0	Disabled — presence_penalty handles it

presence_penalty = 1.5 is the single most load-bearing value in this chapter. Set it everywhere.

FIRST INFERENCE

Run a prompt

```
$ ./build/bin/llama-cli \
    --model ~/models/Qwen3.5-0.8B-Q8_0.gguf \
    --threads 4 --ctx-size 1024 --single-turn \
    --temperature 0.7 --top-p 0.8 --top-k 20 \
    --presence-penalty 1.5 --repeat-penalty 1.0 \
    --reasoning off --reasoning-budget 0 \
    -p "What is the capital of Brazil? One word."
```

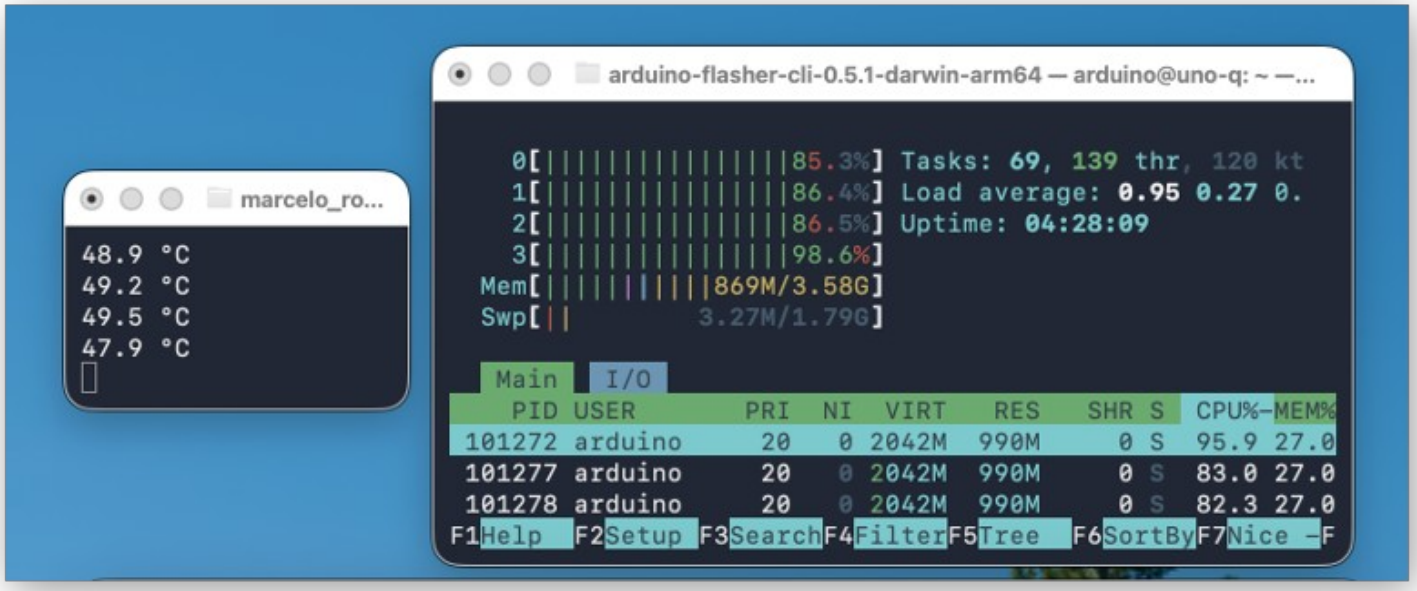
> Expect ~5.6 tokens/s (and ~11 on the prompt). That is the real number on this board — usable, not fast.



First local inference

THERMALS

No heatsink. No fan. ~54 °C.



All four cores at 100%, temperature during inference

> ~30 °C idle → ~54 °C inference → ~62 °C on sustained long answers. Throttle threshold is 70–80 °C.

ERGONOMICS

Wrap it in a script

```
#!/usr/bin/env bash
# ~/qwen_cli_llama.sh

MODEL=~/models/Qwen3.5-0.8B-Q8_0.gguf
LLAMA=~/llama.cpp/build/bin/llama-cli

"$LLAMA" --model "$MODEL" --threads 4 --ctx-size 1024 \
  --temperature 0.7 --top-p 0.8 --top-k 20 \
  --presence-penalty 1.5 --repeat-penalty 1.0 \
  --reasoning off --reasoning-budget 0 -st -p "$*"
```

chmod +x it, add ~/ to PATH in .bashrc, and it becomes: qwen_cli_llama.sh "your prompt"



The script in use

ONE CAVEAT

For anything you build on, use llama-server.

llama-cli is less reliable than the server at suppressing Qwen3.5 reasoning traces — a known issue. Use the CLI for quick experiments; target the server for real work.

STAGE 4 OF 6

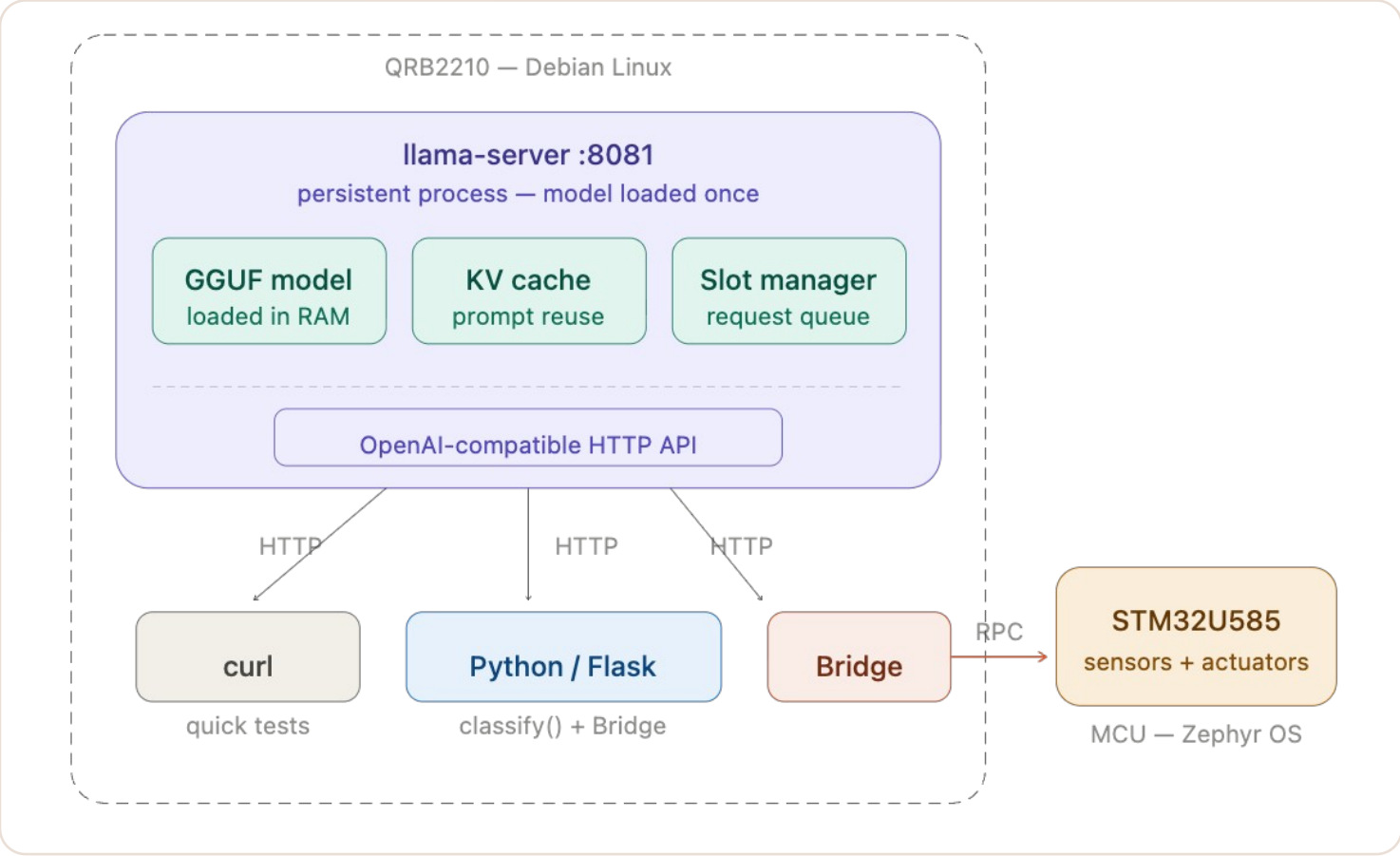
llama-server: the HTTP API

4

Load the GGUF into RAM once, expose an OpenAI-compatible endpoint, query it from anything.

THE SHAPE OF IT

One model in RAM, many clients



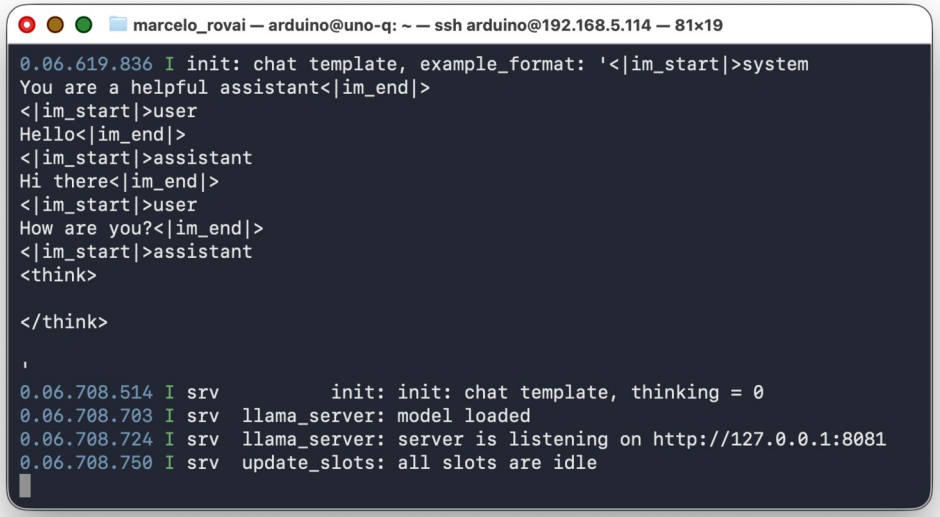
llama-server: load once, serve anything that speaks HTTP

TERMINAL 1

Start the server in the foreground

```
$ ./build/bin/llama-server \
    --model ~/models/Qwen3.5-0.8B-Q8_0.gguf \
    --threads 4  --ctx-size 1024 \
    --port 8081 \
    --reasoning off --reasoning-budget 0 \
    --alias qwen3.5-0.8b
```

Do not close this terminal. The server lives as long as this process. In the log look for: the model loading · thinking = 0 · HTTP server listening.



Terminal 1 — the server log

THE FLAG TO INTERNALISE

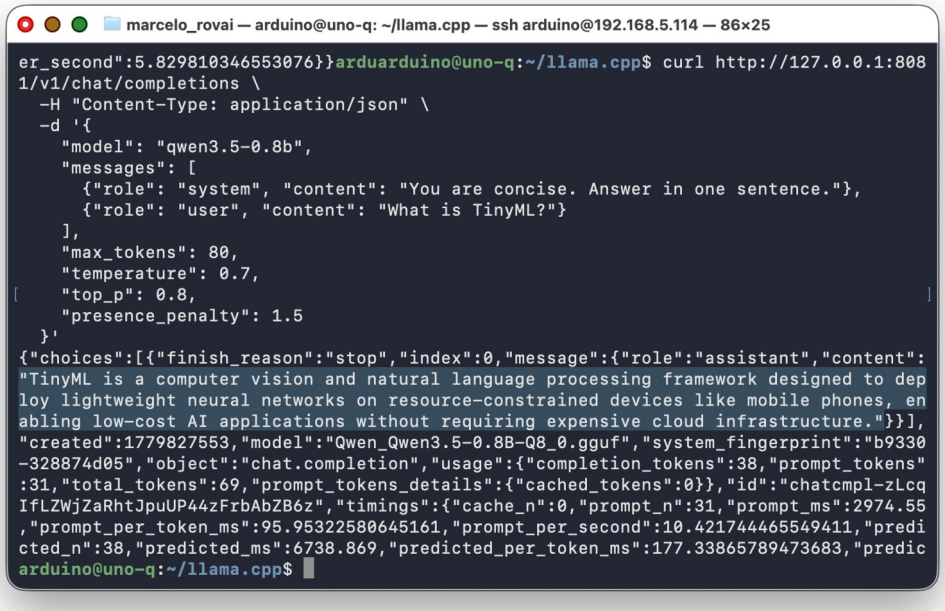
--alias gives the model a stable name.

curl, Python and the WebUI can always ask for "model": "qwen3.5-0.8b", no matter which GGUF file is loaded behind it. That matters the moment you start swapping models — as Chapters 3 and 5 do.

TERMINAL 2

Is it alive?

```
$ curl -s http://127.0.0.1:8081/health | python3 -m json.tool
  {"status": "ok"}
```



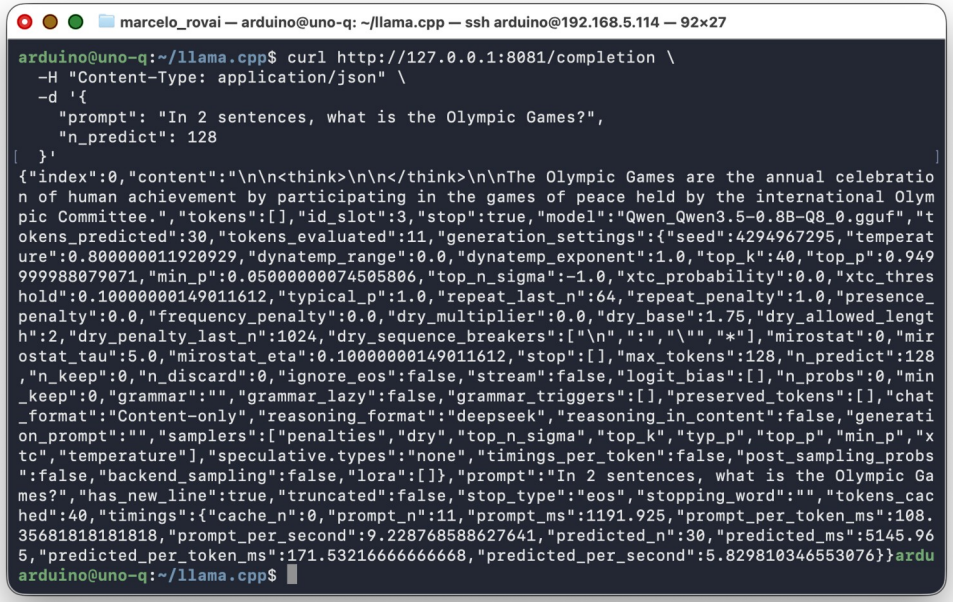
/health

TERMINAL 2

The raw completion endpoint

```
$ curl http://127.0.0.1:8081/completion \
    -H "Content-Type: application/json" \
    -d '{"prompt": "What is TinyML?",
         "n_predict": 128}'
```

> This is llama.cpp's own endpoint — useful, but not the one your code should target.

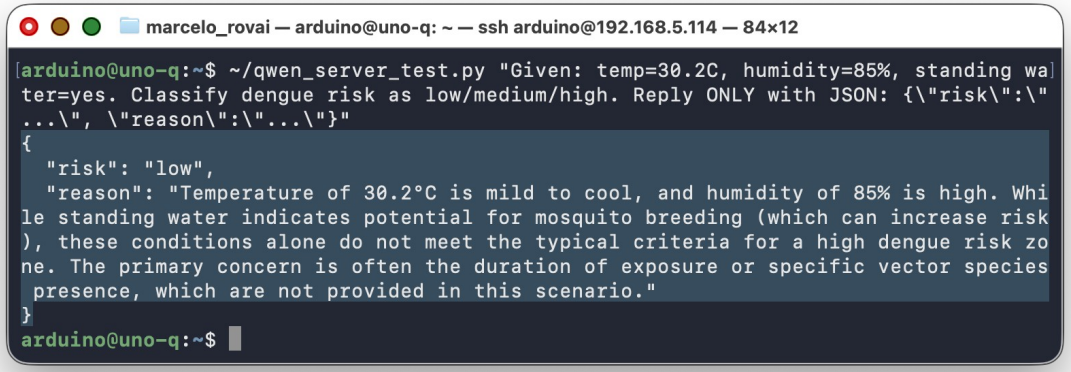


A raw completion

TERMINAL 2

The endpoint that actually matters

```
$ curl http://127.0.0.1:8081/v1/chat/completions \
    -H "Content-Type: application/json" \
    -d '{"model": "qwen3.5-0.8b",
         "messages": [
           {"role":"system","content":"You are concise."},
           {"role":"user","content":"What is TinyML?"}],
         "max_tokens": 80, "presence_penalty": 1.5}'
```

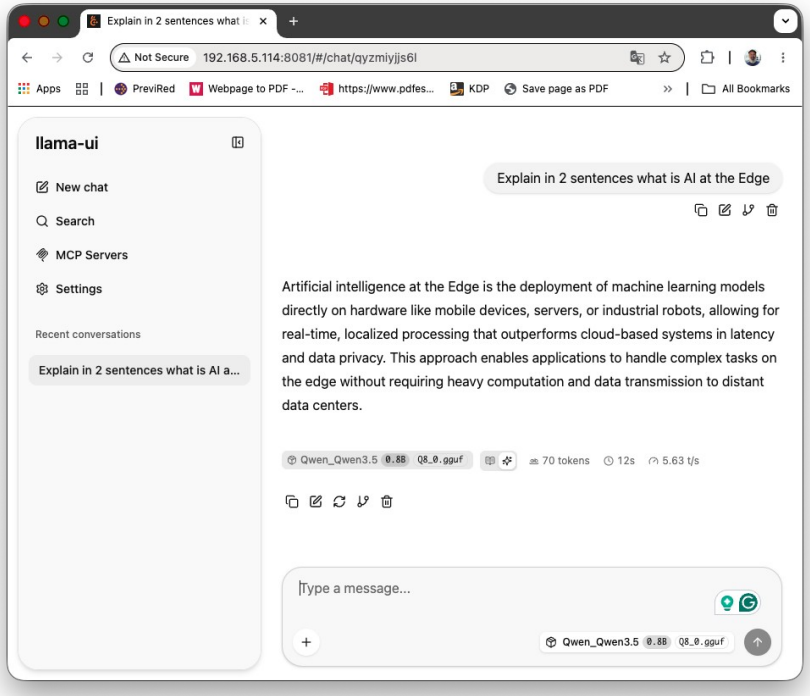


The JSON response

> The answer is in choices[0].message.content . This is the OpenAI-compatible shape every Python client in the next stage will use.

TWO EXTRAS

The WebUI ships inside the binary



llama-server's embedded WebUI, from any device on the Wi-Fi

> Add --host 0.0.0.0 and open http://<UNO_Q_IP>:8081/ — or keep it on loopback and tunnel: ssh -L 8081:127.0.0.1:8081 arduino@<IP>

TWO EXTRAS

Watch the temperature while it works

```
$ watch -n1 cat /sys/class/thermal/thermal_zone0/temp
```

> Divide by 1000 for °C. Run it in a third session during a long generation.



Thermal zone, live

STAGE 5 OF 6

Calling it from Python

5

The standard openai client, then raw HTTP — so you can see what the library does underneath.

THE RECOMMENDED PATH

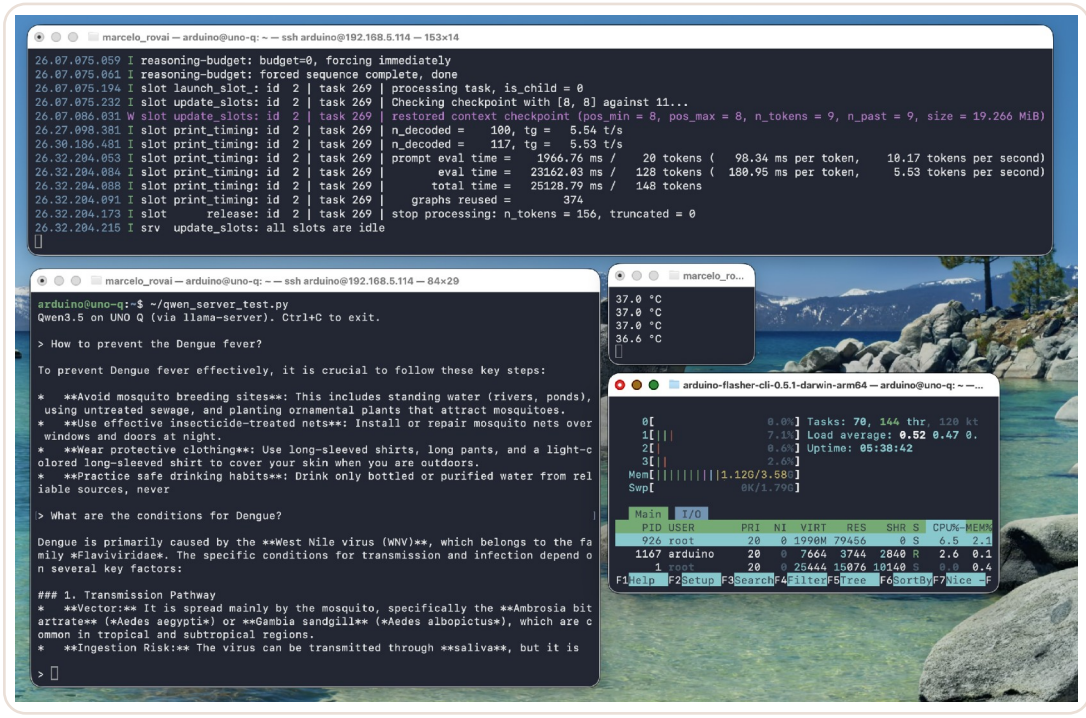
The openai client library

```
$ pip install --user openai

from openai import OpenAI

client = OpenAI(
    base_url="http://127.0.0.1:8081/v1",
    api_key="not-needed")    # ignored, but required

response = client.chat.completions.create(
    model="qwen3.5-0.8b", messages=messages,
    max_tokens=128, stream=True)
```



The client, one-shot and interactive

> llama-server ignores the API key, but the client library insists on one. Any string works.

THREE THINGS TO NOTICE

In that code

1

stream=True

Tokens print as they are generated instead of waiting for the whole response. On a 5 tok/s board this transforms perceived speed.

2

history is a plain list

Each turn appends the prompt and the reply, so the model sees the conversation next call. There is no hidden server-side state.

3

strip_think() is cosmetic

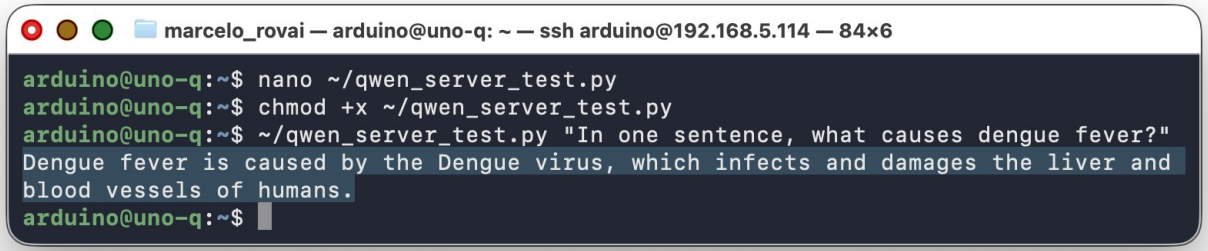
Even with reasoning off, Qwen3.5 sometimes emits an empty <think></think>. The content is empty — no reasoning is happening.

> Multi-turn memory is just a Python list you keep appending to. That is worth saying out loud.

PATTERN 1

Ask for JSON, get JSON

```
$ ~/qwen_client.py \
  "Given: temp=30.2C, humidity=85%,
   standing water=yes.
   Classify dengue risk low/medium/high.
   Reply ONLY with JSON:
   {risk: ..., reason: ...}"
```



Structured output

> Valid JSON most of the time at Q8. When it is not, strip the fences and retry — the parse_verdict() pattern in the written chapter.

PATTERN 2

Or skip the dependency entirely

```
import requests

resp = requests.post(
    "http://127.0.0.1:8081/v1/chat/completions",
    json={"model": "qwen3.5-0.8b",
          "messages": [...],
          "max_tokens": 80,
          "presence_penalty": 1.5},
    timeout=120)
print(resp.json()["choices"][0]["message"]["content"])
```

Useful for an embedded script, a minimal container image, or just to see the wire format. It is plain JSON over HTTP — nothing magic in the library.

CHOOSING

Library or raw HTTP?

	openai library	raw requests
Code	Shorter, typed response objects	More verbose, manual JSON parsing
Streaming	Built-in iterator over chunks	Manual SSE line parsing
Moving to a cloud provider	Change base_url + api_key	Change URL and headers by hand
Dependencies	Adds the openai package	Only requests — often already installed
Error handling	Raises typed exceptions	Inspect status_code and response.text
Best for	Application code, anything long-lived	Quick scripts, debugging, tight environments

> Application code can move from a local SLM to a cloud LLM and back with a one-line URL change. That portability is why we used the OpenAI-compatible surface.

STAGE 6 OF 6

Performance and limits

6

Measured numbers, the failure modes, and where the 0.8B model stops being enough.

MEASURED

What to actually expect

Cold model load	~4.5 s
Idle RAM, server running	~950 MB
Model + context	~1.0–1.05 GB
Prompt processing	~11 tok/s
Generation	~5.6 tok/s
CPU during decode	4 cores @ 100%
Idle temperature	~45 °C
Inference temperature	~55 °C
Throttle threshold	70–80 °C — never reached

5.6

tokens/s generation

3.1 W

maximum power draw

THE OPTIMISATION THAT ACTUALLY PAYS

Cap max_tokens.

People optimise tokens per second when wall-clock latency is what users feel. A 30-token answer at 10 tok/s takes 3 seconds. A 100-token answer at the same rate takes 10. Design prompts to keep answers short — it is the cheapest latency win on this board.

FAILURE MODES

The four things that will go wrong

1

Model loops, or answers forever

Thinking mode is active. Check --reasoning off and presence_penalty 1.5. This accounts for most of it.

2

OOM crash during model load

Confirm swap with free -h and reduce --ctx-size. The load spike is the danger, not steady-state inference.

3

JSON parse failures

Strip fences and a leading 'json' token, then retry. Occasional stray text is normal at this size.

4

Repetition loops

presence_penalty = 1.5, everywhere. A documented behaviour of small Qwen3.5 variants, not a bug in your setup.

> Every one of these has a specific fix. Teach the fix with the symptom and students stop guessing.

SIZING UP

When to move from 0.8B to 2B

Dimension	0.8B Q8_0	2B UD-Q4_K_XL
Model file size	~800 MB	~1.34 GB
RAM footprint (generating)	~1.0 GB	~1.5 GB
Free RAM after load (of ~3 GB)	~2.0 GB	~1.5 GB
Generation speed	~5.6 tok/s	~2.5 tok/s
Quality on structured JSON	Good with a strong few-shot	Noticeably more robust
Quality on free-form chat	Adequate	Meaningfully better

> Parameter count beats quantization: a 4-bit-larger model often outperforms a smaller one at higher precision, with nearly the same memory usage.

BE REALISTIC

What this setup cannot do

PRECISION

Q4 hurts here

A 0.8B model at Q4 loses more than a 7B does — less redundancy to exploit. Quality at this size is uneven: expect occasional nonsense JSON or hallucinated reasons.

HARDWARE

No NPU

The Adreno 702 GPU is not a reliable llama.cpp target. Four A53 cores do all the work. On older images, storage is tight too.

BEHAVIOUR

Thinking is unusable

Multi-minute inference and frequent loops. Always --reasoning off. Keep a human in the loop for anything safety-critical.

But the patterns port upward: the VENTUNO Q — 40 TOPS NPU, 16 GB RAM — makes billions of parameters interactive. Same llama.cpp, same HTTP API. Only the numbers change.

CARRY THESE OUT OF CHAPTER 2

Three things

1

Local generative AI, on Arduino hardware

On the same board used for Blink, a language model now answers with no internet connection.

2

Standard, transferable tools

An OpenAI-compatible API and the official client library. Nothing here is board-specific except the constraints.

3

Bounded — and that is the lesson

Small models, short outputs, batch-rate inference. Cap max_tokens, keep presence_penalty at 1.5, thinking off.

Bounded, but the boundary moves. The VENTUNO Q changes the numbers, not the patterns.

RESOURCES

Where to read further

Resource	Where
`llama.cpp repository`	github.com/ggml-org/llama.cpp
`llama.cpp HTTP server docs`	github.com/ggml-org/llama.cpp/blob/master/tools/server/README.md
`Qwen3.5-0.8B GGUF (Bartowski)`	huggingface.co/bartowski/Qwen_Qwen3.5-0.8B-GGUF
`Qwen3.5-0.8B GGUF (Unsloth Dynamic)`	huggingface.co/unsloth/Qwen3.5-0.8B-GGUF
`Unsloth Qwen3.5 inference guide`	unsloth.ai/docs/models/qwen3.5
`OpenAI Python library`	github.com/openai/openai-python

Next — Ch. 3, Multimodal AI at the Edge: the same model given eyes. A vision projector, a vision-enabled server, and reasoning about camera input.

Questions?

Prof. Marcelo J. Rovai

rovai@unifei.edu.br

UNIFEI — Federal University of Itajubá, Brazil
AiEng4D — Academic Network Co-Chair
EdgeAIP — Academia-Industry Partnership Co-Chair



